

1_1_foundational_models.py

```
# %%
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain.messages import AIMessage, HumanMessage

# temperature
# max tokens
# timeout
# max retries
# https://docs.langchain.com/oss/python/integrations/chat
model = init_chat_model(model="gpt-5-nano")
# %%
response = model.invoke("What's the capital of Moon?")
print(response)
# %%
print(type(response))
print(response.response_metadata)
# %%
agent = create_agent(model=model)

response = agent.invoke(
    {"messages": [HumanMessage(content="What's the capital of the Moon?")]}
)
print(response)
```

```

# %%
print(response["messages"][-1].content)
# %%
response = agent.invoke(
    {
        "messages": [
            HumanMessage(content="What's the capital of the Moon?"),
            AIMessage(content="The capital of the Moon is Luna City."),
            HumanMessage(content="Interesting, tell me more about Luna City"),
        ]
    }
)
print(response)
# %%
print(response["messages"][-1].content)
# %%
for token, metadata in agent.stream(
    {
        "messages": [
            HumanMessage(content="Tell me all about Luna City, the capital of t
he Moon")
        ]
    },
    stream_mode="messages",
):
    # token is a message chunk with token content

```

```
# metadata contains which node produced the token

if token.content: # Check if there's actual content
    print(token.content, end="", flush=True) # Print token
# %%
```

1_1_prompting.py

```
# %%
from langchain.agents import create_agent
from langchain.chat_models import init_chat_model
from langchain.messages import HumanMessage
from pydantic import BaseModel
from pprint import pprint
# temperature
# max tokens
# timeout
# max retries
# https://docs.langchain.com/oss/python/integrations/chat
model = init_chat_model(model="gpt-5-nano")
# %%
system_prompt1 = """
```

You are a science fiction writer, create a space capital city at the users request.

Please keep to the below structure.

Name: The name of the capital city

Location: Where it is based

Vibe: 2-3 words to describe its vibe

Economy: Main industries

```
"""
system_prompt = "You are a science fiction writer, create a capital city at the
  users request."
class CapitalInfo(BaseModel):
    name: str
    location: str
    vibe: str
    economy: str
model = init_chat_model(model="gpt-5-nano", temperature=1.0)

question = HumanMessage(content="What is the capital of The Moon?")
scifi_agent = create_agent(
    model=model,
    system_prompt=system_prompt,
    response_format=CapitalInfo
)

response = scifi_agent.invoke({"messages": [question]})
pprint(response)
# %%
pprint(response['messages'][-1].content)
```

1_2_tools.py

```
# %%
from langchain.tools import tool
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from pprint import pprint

load_dotenv()

# %%
# @tool
# def square_root(x: float) -> float:
#     """Calculate the square root of a number"""
#     return x**0.5

# @tool("square_root")
# def tool1(x: float) -> float:
#     """Calculate the square root of a number"""
#     return x**0.5

@tool("square_root", description="Calculate the square root of a number")
def tool1(x: float) -> float:
    return x**0.5
```

```
tool1.invoke({"x": 467})
# %%
agent = create_agent(
    model="gpt-5-nano",
    tools=[tool1],
    system_prompt="You are an arithmetic wizard. Use your tools to calculate the square root and square of any number.",
)

question = HumanMessage(content="What is the square root of 467?")

response = agent.invoke({"messages": [question]})

print(response["messages"][-1].content)
# %%
pprint(response["messages"])
# %%
print(response["messages"][1].tool_calls)

# %%
```

1_2_web_search.py

```
# %%
from pprint import pprint
from typing import Any, Dict

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain.tools import tool
from tavily import TavilyClient

load_dotenv()

tavily_client = TavilyClient()

@tool
def web_search(query: str) -> Dict[str, Any]:
    """Search the web for information"""
    return tavily_client.search(query)

# Optional quick local test:
# result = web_search.invoke("Who is the current mayor of San Francisco?")
# print(result)
```

```
agent = create_agent(  
    model="gpt-5-nano",  
    tools=[web_search],  
)  
  
question = HumanMessage(content="Who is the current mayor of San Francisco?")  
response = agent.invoke({"messages": [question]})  
  
print(response["messages"][-1].content)  
pprint(response["messages"])  
# %%
```

1_3_memory.py

```
# %%
from pprint import pprint
from dotenv import load_dotenv
from langgraph.checkpoint.memory import InMemorySaver
from langchain.agents import create_agent
from langchain.messages import HumanMessage

load_dotenv()

agent = create_agent(
    model="gpt-5-nano",
    checkpointer=InMemorySaver()
)
config = {"configurable": {"thread_id": "1"}}
question = HumanMessage(content="This is Sunny and I like black color")
response = agent.invoke({"messages": [question]}, config=config)

print(response["messages"][-1].content)
# %%
question1 = HumanMessage(content="What is my name and what color do I like?")
response = agent.invoke({"messages": [question1]}, config=config)
pprint(response["messages"])
# %%
```

1_4_multimodal_messages.py

```
# %%
from pprint import pprint
from typing import Any, Dict

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain.tools import tool
from tavily import TavilyClient

load_dotenv()

tavily_client = TavilyClient()

@tool
def web_search(query: str) -> Dict[str, Any]:
    """Search the web for information"""
    return tavily_client.search(query)

# Optional quick local test:
# result = web_search.invoke("Who is the current mayor of San Francisco?")
```

```
# print(result)

agent = create_agent(
    model="gpt-5-nano",
    tools=[web_search],
)

question = HumanMessage(content="Who is the current mayor of San Francisco?")
response = agent.invoke({"messages": [question]})

pprint(response["messages"])
# %%
print(response["messages"][-1].content)
```

1_5_personal_chef.py

```
# %%
from dotenv import load_dotenv
from langchain.tools import tool
from typing import Dict, Any
from tavily import TavilyClient
from langchain.agents import create_agent
from langchain.messages import HumanMessage

load_dotenv()

tavily_client = TavilyClient()

@tool
def web_search(query: str) -> Dict[str, Any]:

    """Search the web for information"""

    return tavily_client.search(query)

system_prompt = """

You are a personal chef. The user will give you a list of ingredients they have
left over in their house.
```

Using the web search tool, search the web for recipes that can be made with the ingredients they have.

Return recipe suggestions and eventually the recipe instructions to the user, if requested.

```
"""
```

```
agent = create_agent(  
    model="gpt-5-nano",  
    tools=[web_search],  
    system_prompt=system_prompt  
)
```

```
response = agent.invoke({"messages": [HumanMessage(content="I have chicken, rice, and broccoli. What can I make for dinner?")]}))  
print(response)  
# %%
```

2_1_mcp.py

```
# %%  
from dotenv import load_dotenv  
  
load_dotenv()  
  
# MCP Host can also be an Agent or Chat Application. MCP Client resides in MCP  
# Host. MCP Client connect to MCP Server  
# LangChain or LangGraph is a Vendor-Agnostic Framework. A vendor-agnostic fram  
# ework is a design approach that ensures systems, software, or hardware operate  
# independently of any specific vendor, preventing vendor lock-in and promoting i  
# nteroperability.  
# 3rd party MCP Servers are availble at mcp.so  
# REFERENCES: To dig deeper into MCP, check out https://learn.deeplearning.ai/c  
# ourses/mcp-build-rich-context-ai-apps-with-anthropic/information  
  
# %%  
import sys  
import asyncio  
  
# Fix for Windows issues in Jupyter notebooks  
# if sys.platform == "win32":  
#     # 1. Use ProactorEventLoop for subprocess support  
#     if not isinstance(asyncio.get_event_loop_policy(), asyncio.WindowsProacto  
#     rEventLoopPolicy):
```

```

#         asyncio.set_event_loop_policy(asyncio.WindowsProactorEventLoopPolicy(
#     ))

#     # 2. Redirect stderr to avoid fileno() error when launching MCP servers
#     if "ipykernel" in sys.modules:
#         sys.stderr = sys.__stderr__

# ## Local MCP server

# %%
from langchain_mcp_adapters.client import MultiServerMCPClient

client = MultiServerMCPClient(
    {
        "local_server": {
            "transport": "stdio",
            "command": "python",
            "args": ["2_1_mcp_server.py"],
        }
    }
)

# %%
# get tools
tools = await client.get_tools()

```

```

# get resources
resources = await client.get_resources("local_server")

# get prompts
prompt = await client.get_prompt("local_server", "prompt")
prompt = prompt[0].content

# %%
from langchain.agents import create_agent

agent = create_agent(
    model="gpt-5-nano",
    tools=tools,
    system_prompt=prompt
)

# %%
from langchain.messages import HumanMessage

config = {"configurable": {"thread_id": "1"}}

response = await agent.ainvoke(
    {"messages": [HumanMessage(content="Tell me about the langchain-mcp-adapter
s library")]},
    config=config
)

```

```

# %%
from pprint import pprint

pprint(response)

# ## Online MCP

# %%
client = MultiServerMCPClient(
    {
        "time": {
            "transport": "stdio",
            "command": "uvx",
            "args": [
                "mcp-server-time",
                "--local-timezone=America/New_York"
            ]
        }
    }
)

tools = await client.get_tools()

# %%
agent = create_agent(

```

```

    model="gpt-5-nano",
    tools=tools,
)

# %%
question = HumanMessage(content="What time is it?")

response = await agent.ainvoke(
    {"messages": [question]}
)

pprint(response)

# %%
# Above code only works in Jupyter notebooks. For running in a Python script, please refer below.
# import asyncio
# from pprint import pprint

# from dotenv import load_dotenv
# from langchain.agents import create_agent
# from langchain.messages import HumanMessage
# from langchain_mcp_adapters.client import MultiServerMCPClient

# load_dotenv()

```

```

# # MCP Host can also be an Agent or Chat Application. MCP Client resides in MC
P Host. MCP Client connect to MCP Server
# # LangChain or LangGraph is a Vendor-Agnostic Framework. A vendor-agnostic fr
amework is a design approach that ensures systems, software, or hardware operat
e independently of any specific vendor, preventing vendor lock-in and promoting
interoperability.
# # 3rd party MCP Servers are availble at mcp.so
# # REFERENCES: To dig deeper into MCP, check out https://learn.deeplearning.ai/courses/mcp-build-rich-context-ai-apps-with-anthropic/information

# # Fix for Windows issues in Jupyter notebooks
# # if sys.platform == "win32":
# #     # 1. Use ProactorEventLoop for subprocess support
# #     if not isinstance(asyncio.get_event_loop_policy(), asyncio.WindowsProac
torEventLoopPolicy):
# #         asyncio.set_event_loop_policy(asyncio.WindowsProactorEventLoopPolic
y())

# #     # 2. Redirect stderr to avoid fileno() error when launching MCP servers
# #     if "ipykernel" in sys.modules:
# #         sys.stderr = sys.__stderr__

# async def main():
#     # ## Local MCP server

```

```

# client = MultiServerMCPClient(
#     {
#         "local_server": {
#             "transport": "stdio",
#             "command": "python",
#             "args": ["2_1_mcp_server.py"],
#         }
#     }
# )

# # get tools
# tools = await client.get_tools()

# # get resources
# resources = await client.get_resources("local_server")

# # get prompts
# prompt = await client.get_prompt("local_server", "prompt")
# prompt = prompt[0].content

# agent = create_agent(model="gpt-5-nano", tools=tools, system_prompt=prompt)

# config = {"configurable": {"thread_id": "1"}}

# response = await agent.ainvoke(

```

```

#         {
#             "messages": [
#                 HumanMessage(content="Tell me about the langchain-mcp-adapter
s library")
#             ]
#         },
#         config=config,
#     )

#     pprint(response)

#     # ## Online MCP

#     client = MultiServerMCPClient(
#         {
#             "time": {
#                 "transport": "stdio",
#                 "command": "uvx",
#                 "args": ["mcp-server-time", "--local-timezone=America/New_Yor
k"],
#             }
#         }
#     )

#     tools = await client.get_tools()

```

```
# agent = create_agent(  
#     model="gpt-5-nano",  
#     tools=tools,  
# )  
# question = HumanMessage(content="What time is it?")  
# response = await agent.ainvoke({"messages": [question]})  
  
# pprint(response)  
  
# if __name__ == "__main__":  
#     asyncio.run(main())
```

2_1_mcp_localmcp.py

```
# %%
import asyncio
from pprint import pprint

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain_mcp_adapters.client import MultiServerMCPClient
from tavily import TavilyClient

load_dotenv()

# MCP Host can also be an Agent or Chat Application. MCP Client resides in MCP
# Host. MCP Client connect to MCP Server
# LangChain or LangGraph is a Vendor-Agnostic Framework. A vendor-agnostic framework
# is a design approach that ensures systems, software, or hardware operate
# independently of any specific vendor, preventing vendor lock-in and promoting interoperability.
# 3rd party MCP Servers are available at mcp.so
# REFERENCES: To dig deeper into MCP, check out https://learn.deeplearning.ai/courses/mcp-build-rich-context-ai-apps-with-anthropic/information

# %%
```

```

load_dotenv()

tavily_client = TavilyClient()

client = MultiServerMCPClient(
    {
        "local_server": {
            "transport": "stdio",
            "command": "python",
            "args": ["2_1_mcp_server.py"],
        }
    }
)

# %%
async def main():
    tools = await client.get_tools()
    resources = await client.get_resources("local_server")
    prompt = await client.get_prompt("local_server", "prompt")
    prompt = prompt[0].content

    agent = create_agent(model="gpt-5-nano", tools=tools, system_prompt=prompt)

    config = {"configurable": {"thread_id": "1"}}

```

```
response = await agent.ainvoke(
    {
        "messages": [
            HumanMessage(content="Tell me about the langchain-mcp-adapters
library")
        ]
    },
    config=config,
)

pprint(response)

asyncio.run(main())
```

2_1_mcp_remotemcp.py

```
# %%
from pprint import pprint

from dotenv import load_dotenv

load_dotenv()

# MCP Host can also be an Agent or Chat Application. MCP Client resides in MCP
# Host. MCP Client connect to MCP Server
# LangChain or LangGraph is a Vendor-Agnostic Framework. A vendor-agnostic fram
# ework is a design approach that ensures systems, software, or hardware operate
# independently of any specific vendor, preventing vendor lock-in and promoting i
# nteroperability.
# 3rd party MCP Servers are availble at mcp.so
# REFERENCES: To dig deeper into MCP, check out https://learn.deeplearning.ai/courses/mcp-build-rich-context-ai-apps-with-anthropic/information

import asyncio
import sys

# %%
from langchain_mcp_adapters.client import MultiServerMCPClient

# ## Online MCP
```

```

client = MultiServerMCPClient(
    {
        "time": {
            "transport": "stdio",
            "command": "uvx",
            "args": ["mcp-server-time", "--local-timezone=Asia/Kolkata"],
        }
    }
)

# %%
# get tools
tools = await client.get_tools()

# %%
from langchain.agents import create_agent

agent = create_agent(model="gpt-5-nano", tools=tools)

pprint(tools)
# %%
agent = create_agent(
    model="gpt-5-nano",
    tools=tools,
)

```

```
# %%  
from langchain.messages import HumanMessage  
  
question = HumanMessage(content="What time is it?")  
  
response = await agent.ainvoke({"messages": [question]})  
  
pprint(response)  
  
# %%
```

2_1_mcp_server.py

```
from dotenv import load_dotenv

load_dotenv()

from mcp.server.fastmcp import FastMCP
from tavily import TavilyClient
from typing import Dict, Any
from requests import get

mcp = FastMCP("mcp_server")

tavily_client = TavilyClient()

# Tool for searching the web
@mcp.tool()
def search_web(query: str) -> Dict[str, Any]:
    """Search the web for information"""

    results = tavily_client.search(query)

    return results
```

```

# Resources - provide access to langchain-ai repo files
@mcp.resource("github://langchain-ai/langchain-mcp-adapters/blob/main/README.md")
def github_file():
    """
    Resource for accessing langchain-ai/langchain-mcp-adapters/README.md file

    """
    url = f"https://raw.githubusercontent.com/langchain-ai/langchain-mcp-adapters/blob/main/README.md"
    try:
        resp = get(url)
        return resp.text
    except Exception as e:
        return f"Error: {str(e)}"

# Prompt template
@mcp.prompt()
def prompt():
    """Analyze data from a langchain-ai repo file with comprehensive insights"""
    """
    You are a helpful assistant that answers user questions about LangChain, LangGraph and LangSmith.
    """

```

You can use the following tools/resources to answer user questions:

- search_web: Search the web for information
- github_file: Access the langchain-ai repo files

If the user asks a question that is not related to LangChain, LangGraph or LangSmith, you should say "I'm sorry, I can only answer questions about LangChain, LangGraph and LangSmith."

You may try multiple tool and resource calls to answer the user's question.

You may also ask clarifying questions to the user to better understand their question.

"""

```
if __name__ == "__main__":  
    mcp.run(transport="stdio")
```

2_1_travel_agent.py

```
import asyncio
from pprint import pprint

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain_mcp_adapters.client import MultiServerMCPClient
from langgraph.checkpoint.memory import InMemorySaver

load_dotenv()

async def main():
    client = MultiServerMCPClient(
        {
            "travel_server": {
                "transport": "streamable_http",
                "url": "https://mcp.kiwi.com",
            }
        }
    )

    tools = await client.get_tools()

    agent = create_agent(
```

```

        "gpt-5-nano",
        tools=tools,
        checkpointer=InMemorySaver(),
        system_prompt="You are a travel agent. No follow up questions.",
    )

    config = {"configurable": {"thread_id": "1"}}

    response = await agent.ainvoke(
        {
            "messages": [
                HumanMessage(
                    content="Get me a direct flight from San Francisco to Tokyo
on March 31st"
                )
            ]
        },
        config,
    )

    pprint(response)
    print()
    pprint(response["messages"][-1].content)

if __name__ == "__main__":

```


2_2_state_1.py

```
# %%
from pprint import pprint

from dotenv import load_dotenv
from langchain.agents import AgentState, create_agent
from langchain.messages import HumanMessage, ToolMessage
from langchain.tools import ToolRuntime, tool
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

load_dotenv()

# %%
class CustomState(AgentState):
    favourite_colour: str

# ## Write to state
# It just updates the state using a update tool. And ad

# %%
@tool
```

```

def update_favourite_colour(favourite_colour: str, runtime: ToolRuntime) -> Command:
    """Update the favourite colour of the user in the state once they've revealed it."""
    return Command(
        update={
            "favourite_colour": favourite_colour,
            "messages": [
                ToolMessage(
                    "Successfully updated favourite colour",
                    tool_call_id=runtime.tool_call_id,
                )
            ],
        }
    )

# %%
agent = create_agent(
    "gpt-5-nano",
    tools=[update_favourite_colour],
    checkpointer=InMemorySaver(),
    state_schema=CustomState,
)

```

```
# %%  
response = agent.invoke(  
    {"messages": [HumanMessage(content="My favourite colour is red")]},  
    {"configurable": {"thread_id": "1"}},  
)  
  
# %%  
pprint(response)
```

2_2_state_2.py

```
# %%
from pprint import pprint

from dotenv import load_dotenv
from langchain.agents import AgentState, create_agent
from langchain.messages import HumanMessage, ToolMessage
from langchain.tools import ToolRuntime, tool
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

load_dotenv()
# %%
class CustomState(AgentState):
    favourite_colour: str

# ## Write / update the state
@tool
def update_favourite_colour(favourite_colour: str, runtime: ToolRuntime) -> Command:
    """Update the favourite colour of the user in the state once they've revealed it."""
    return Command(update={
        "favourite_colour": favourite_colour,
        "messages": [ToolMessage("Successfully updated favourite colour", tool_
```

```

call_id=runtime.tool_call_id)]]}
    )

# ## Read state
@tool
def read_favourite_colour(runtime: ToolRuntime) -> str:
    """Read the favourite colour of the user from the state."""
    try:
        return runtime.state["favourite_colour"]
    except KeyError:
        return "No favourite colour found in state"

# %%
agent = create_agent(
    "gpt-5-nano",
    tools=[update_favourite_colour, read_favourite_colour],
    checkpointer=InMemorySaver(),
    state_schema=CustomState,
)

# %%
response = agent.invoke(
    {"messages": [HumanMessage(content="My favourite colour is green")]},
    {"configurable": {"thread_id": "1"}},
)

```

```
pprint(response)
pprint('-----')
# %%
response = agent.invoke(
    {"messages": [HumanMessage(content="What's my favourite colour?")]}},
    {"configurable": {"thread_id": "1"}}},
)

pprint(response)
# %%
```

2_2_0_runtime_context_wontwork.py

```
# %%
from dataclasses import dataclass
from pprint import pprint

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain.tools import ToolRuntime, tool

load_dotenv()

# %%
@dataclass
class ColourContext:
    favourite_colour: str = "blue"
    least_favourite_colour: str = "yellow"

agent = create_agent(model="gpt-5-nano", context_schema=ColourContext)

# This will not work as the agent does not have access to the context when invoking without tools
response = agent.invoke(
```

```
    {"messages": [HumanMessage(content="What is my favourite colour?")]},  
    context=ColourContext(),  
)  
  
pprint(response)
```

2_2_1_runtime_context.py

```
# %%
from dataclasses import dataclass
from pprint import pprint

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain.tools import ToolRuntime, tool

load_dotenv()

# %%
@dataclass
class ColourContext:
    favourite_colour: str = "blue"
    least_favourite_colour: str = "yellow"

# ## Accessing Context
# %%
@tool
def get_favourite_colour(runtime: ToolRuntime) -> str:
    """Get the favourite colour of the user"""
    return runtime.context.favourite_colour
```

```

@tool
def get_least_favourite_colour(runtime: ToolRuntime) -> str:
    """Get the least favourite colour of the user"""
    return runtime.context.least_favourite_colour

# %%
agent = create_agent(
    model="gpt-5-nano",
    tools=[get_favourite_colour, get_least_favourite_colour],
    context_schema=ColourContext,
)

response = agent.invoke(
    {"messages": [HumanMessage(content="What is my favourite colour?")]}},
    # context=ColourContext(),
    context={"favourite_colour": "red", "least_favourite_colour": "green"},
)

pprint(response)

# %%
response = agent.invoke(
    {"messages": [HumanMessage(content="What is colour that I don't like?")]}},
    context=ColourContext(least_favourite_colour="green")

```

```
)
```

```
pprint(response)
```

```
# %%
```

2_3_multi_agent.py

```
# %%
from pprint import pprint
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from langchain.tools import tool
load_dotenv()
# ## Creating subagents
# %%
@tool
def square_root(x: float) -> float:
    """Calculate the square root of a number"""
    return x**0.5

@tool
def square(x: float) -> float:
    """Calculate the square of a number"""
    return x**2

# %%
# create subagents
subagent_1 = create_agent(model="gpt-5-nano", tools=[square_root])
subagent_2 = create_agent(model="gpt-5-nano", tools=[square])
```

```

# %%
@tool
def call_subagent_1(x: float) -> float:
    """Call subagent 1 in order to calculate the square root of a number"""
    response = subagent_1.invoke(
        {"messages": [HumanMessage(content=f"Calculate the square root of {x}")]
    })
    return response["messages"][-1].content

@tool
def call_subagent_2(x: float) -> float:
    """Call subagent 2 in order to calculate the square of a number"""
    response = subagent_2.invoke(
        {"messages": [HumanMessage(content=f"Calculate the square of {x}")]})
    return response["messages"][-1].content

## Creating the main agent
main_agent = create_agent(
    model="gpt-5-nano",
    tools=[call_subagent_1, call_subagent_2],
    system_prompt="You are a helpful assistant who can call subagents to calculate the square root or square of a number.",
)

```

```
# ## Test
# %%
question = "What is the square root of 456?"
response = main_agent.invoke({"messages": [HumanMessage(content=question)]})
pprint(response)
# %%
```

2_4_wedding_planners.py

```
# %%  
from dotenv import load_dotenv  
  
load_dotenv()  
  
# # 2.4 Wedding Planner  
# In this lab you will build a multi-agent wedding planner.  
#  
# > Note: This lab has been updated since filming to make the agent performance  
# more robust to errors and to limit run time. In particular: 1) added error han  
# dling for MCP failures, 2) limited the number of searches. It's worth noting th  
# at, where possible, tool errors should be returned to the agent rather than fai  
# ling so that the agent can adjust its invocation and try again.  
  
# ## Setup Tools  
  
# %%  
import asyncio  
  
from langchain_mcp_adapters.client import MultiServerMCPClient  
from mcp.shared.exceptions import McpError  
from mcp.types import CallToolResult, TextContent  
  
RETRYABLE_MCP_CODES = {-32603}
```

```

class RetryMCPInterceptor:
    """Intercept MCP tool calls: retry transient failures, surface all errors gracefully.

    - Retryable McpError codes (e.g. -32603): retry with exponential backoff.
    - Non-retryable McpError codes (e.g. -32602): return error message immediately.
    - Any other exception (fetch failed, network errors, etc.): retry then return error message.
    """

    def __init__(self, max_retries: int = 3):
        self.max_retries = max_retries

    async def __call__(self, request, handler):
        last_error = None
        for attempt in range(self.max_retries):
            try:
                return await handler(request)
            except McpError as exc:
                last_error = exc
                print(f"[MCP interceptor] {type(exc).__name__} on {request.name} "
                    f"(code {exc.error.code}, attempt {attempt+1}/{self.max_retries}): {exc}")

```

```

        if exc.error.code not in RETRYABLE_MCP_CODES:
            return CallToolResult(
                content=[TextContent(type="text", text=f"Tool call failed (non-retryable): {exc}")],
                isError=False,
            )
        except Exception as exc:
            last_error = exc
            print(f"[MCP interceptor] {type(exc).__name__} on {request.name} "
                  f"(attempt {attempt+1}/{self.max_retries}): {exc}")

            if attempt < self.max_retries - 1:
                await asyncio.sleep(2 ** attempt)

            print(f"[MCP interceptor] all {self.max_retries} retries exhausted for {request.name}")
            return CallToolResult(
                content=[TextContent(type="text", text=f"Tool call failed after {self.max_retries} attempts: {last_error}")],
                isError=False,
            )

client = MultiServerMCPClient(
    {
        "travel_server": {

```

```

        "transport": "streamable_http",
        "url": "https://mcp.kiwi.com"
    }
},
tool_interceptors=[RetryMCPInterceptor()],
)

tools = await client.get_tools()

# %%
from typing import Dict, Any
from tavily import TavilyClient
from langchain.tools import tool

tavily_client = TavilyClient()

@tool
def web_search(query: str, search_number: int, max_search_number: int) -> Dict[
    str, Any]:
    """Search the web for information. You must track your search count by providing
    search_number (starting at 1) and max_search_number on every call.
    Queries must use only plain text characters. Do not use accented or special
    characters
    (e.g., use 'capacite' instead of 'capacit^').
    """

```

```

    if search_number > max_search_number:
        return {"message": "Search limit reached. Please summarize your findings and provide your final answer."}
    try:
        return tavily_client.search(query)
    except Exception as e:
        return {"error": str(e)}

# %%
from langchain_community.utilities import SQLDatabase

db = SQLDatabase.from_uri("sqlite:///resources/Chinook.db")

@tool
def query_playlist_db(query: str) -> str:

    """Query the database for playlist information"""

    try:
        return db.run(query)
    except Exception as e:
        return f"Error querying database: {e}"

# ## Create State

# %%

```

```

from langchain.agents import AgentState

class WeddingState(AgentState):
    origin: str
    destination: str
    guest_count: str
    genre: str

# ## Create Subagents

# %%
from langchain.agents import create_agent

# Travel agent
travel_agent = create_agent(
    model="gpt-5-nano",
    tools=tools,
    system_prompt="""
    You are a travel agent. Search for flights to the desired destination wedding location.
    You are not allowed to ask any more follow up questions, you must find the best flight options based on the following criteria:
    - Price (lowest, economy class)
    - Duration (shortest)
    - Date (time of year which you believe is best for a wedding at this location)
    """
)

```

To make things easy, only look for one ticket, one way.

You may need to make multiple searches to iteratively find the best options

.

You will be given no extra information, only the origin and destination. It is your job to think critically about the best options.

If the MCP tool fails, returns malformed output, or does not give you usable flight results, try the tool again.

Once you have found the best options, let the user know your shortlist of options.

"""

)

%%

Venue agent

```
venue_agent = create_agent(  
    model="gpt-5-nano",  
    tools=[web_search],  
    system_prompt="""
```

You are a venue specialist. Search for venues in the desired location, and with the desired capacity.

You are not allowed to ask any more follow up questions, you must find the best venue options based on the following criteria:

- Price (lowest)
- Capacity (exact match)
- Reviews (highest)

You may need to make multiple searches to iteratively find the best options

```

    You have a suggested limit of 12 web searches. Count every web_search call
you make.
    After 12 searches, you should stop searching and summarize the best options
you have
    found so far.
    """
)

# %%
# Playlist agent
playlist_agent = create_agent(
    model="gpt-5-nano",
    tools=[query_playlist_db],
    system_prompt="""
    You are a playlist specialist. Query the sql database and curate the perfec
t playlist for a wedding given a genre.
    Once you have your playlist, calculate the total duration and cost of the p
laylist, each song has an associated price.
    If you run into errors when querying the database, try to fix them by makin
g changes to the query.
    Do not come back empty handed, keep trying to query the db until you find a
list of songs.

    This is a SQLite database. Before writing any data queries, first discover
the schema.

```

```

    """
)

# ## Main Coordinator

# %%
from langchain.tools import ToolRuntime
from langchain.messages import HumanMessage, ToolMessage
from langgraph.types import Command

@tool
async def search_flights(runtime: ToolRuntime) -> str:
    """Travel agent searches for flights to the desired destination wedding location."""
    origin = runtime.state["origin"]
    destination = runtime.state["destination"]
    response = await travel_agent.ainvoke({"messages": [HumanMessage(content=f"Find flights from {origin} to {destination}")]})
    return response['messages'][-1].content

@tool
def search_venues(runtime: ToolRuntime) -> str:
    """Venue agent chooses the best venue for the given location and capacity."""
    destination = runtime.state["destination"]
    capacity = runtime.state["guest_count"]

```

```

query = f"Find wedding venues in {destination} for {capacity} guests"
response = venue_agent.invoke({"messages": [HumanMessage(content=query)]})
return response['messages'][-1].content

```

@tool

```

def suggest_playlist(runtime: ToolRuntime) -> str:
    """Playlist agent curates the perfect playlist for the given genre."""
    genre = runtime.state["genre"]
    query = f"Find {genre} tracks for wedding playlist"
    response = playlist_agent.invoke({"messages": [HumanMessage(content=query)]
})
    return response['messages'][-1].content

```

@tool

```

def update_state(origin: str, destination: str, guest_count: str, genre: str, runtime: ToolRuntime) -> str:
    """Update the state when you know all of the values: origin, destination, guest_count, genre.
    This tool must be called alone, without any other tool calls. It must complete and return to make,
    the information available to other tools."""
    return Command(update={
        "origin": origin,
        "destination": destination,
        "guest_count": guest_count,
        "genre": genre,

```

```

        "messages": [ToolMessage("Successfully updated state", tool_call_id=run
time.tool_call_id)]]}
    )

# %%
from langchain.agents import create_agent

coordinator = create_agent(
    model="gpt-5-nano",
    tools=[search_flights, search_venues, suggest_playlist, update_state],
    state_schema=WeddingState,
    system_prompt="""
You are a wedding coordinator.
First find all the information you need to update the state. When you have
the information, update the state.
Once that has completed and returned, you can delegate the tasks
to your specialists for flights, venues, and playlists.
Once you have received their answers, coordinate the perfect wedding for me
.
    """
)

# ## Test

# %%
from langchain.messages import HumanMessage

```

```

response = await coordinator.ainvoke(
    {
        "messages": [HumanMessage(content="I'm from London and I'd like a wedding in Paris for 100 guests, jazz-genre")],
    },
    config={"tags": ["WP"], "recursion_limit": 40}, #tag traces to make them easy to find in Langsmith. Increase number of steps the agent can take to 40.
)

# %%
from pprint import pprint

pprint(response)

# %%
print(response["messages"][-1].content)

# link to trace: https://smith.langchain.com/public/7b5fe668-d3e3-4af4-b513-a8cacc0c9e84/r

```

3_2_managing_messages.py

```
from dotenv import load_dotenv
from langchain.agents import AgentState, create_agent
from langchain.agents.middleware import SummarizationMiddleware, before_agent
from langchain.messages import AIMessage, HumanMessage, RemoveMessage, ToolMessage
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.runtime import Runtime
from pprint import pprint
from typing import Any
```

```
load_dotenv()
```

```
## Summarize messages
```

```
# %%
```

```
agent = create_agent(
    model="gpt-5-nano",
    checkpointer=InMemorySaver(),
    middleware=[
        SummarizationMiddleware(
            model="gpt-4o-mini",
            trigger=("tokens", 100),
            keep=("messages", 1)
```

```

    )
],
)

# %%
response = agent.invoke(
    {"messages": [
        HumanMessage(content="What is the capital of the moon?"),
        AIMessage(content="The capital of the moon is Lunapolis."),
        HumanMessage(content="What is the weather in Lunapolis?"),
        AIMessage(content="Skies are clear, with a high of 120C and a low of -1
00C."),
        HumanMessage(content="How many cheese miners live in Lunapolis?"),
        AIMessage(content="There are 100,000 cheese miners living in Lunapolis.
"),
        HumanMessage(content="Do you think the cheese miners' union will strike
?"),
        AIMessage(content="Yes, because they are unhappy with the new president
."),
        HumanMessage(content="If you were Lunapolis' new president how would yo
u respond to the cheese miners' union?"),
    ]},
    {"configurable": {"thread_id": "1"}}
)

pprint(response)

```

```

# %%
print(response["messages"][0].content)

# ## Trim/delete messages

@before_agent
def trim_messages(state: AgentState, runtime: Runtime) -> dict[str, Any] | None
:
    """Remove all the tool messages from the state"""
    messages = state["messages"]

    tool_messages = [m for m in messages if isinstance(m, ToolMessage)]

    return {"messages": [RemoveMessage(id=m.id) for m in tool_messages]}

# %%
agent = create_agent(
    model="gpt-5-nano",
    checkpointer=InMemorySaver(),
    middleware=[trim_messages],
)

# %%
response = agent.invoke(
    {"messages": [

```

```

        HumanMessage(content="My device won't turn on. What should I do?"),
        ToolMessage(content="blorp-x7 initiating diagnostic pingâ f", tool_call_id="1"),
        AIMessage(content="Is the device plugged in and turned on?"),
        HumanMessage(content="Yes, it's plugged in and turned on."),
        ToolMessage(content="temp=42C voltage=2.9v â f greeble complete.", tool_call_id="2"),
        AIMessage(content="Is the device showing any lights or indicators?"),
        HumanMessage(content="What's the temperature of the device?")
    ]},
    {"configurable": {"thread_id": "2"}}
)

pprint(response)

# %%
print(response["messages"][-1].content)

# %%

```

3_3_hitl.py

```
from dotenv import load_dotenv
from langchain.agents import create_agent, AgentState
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langchain.messages import HumanMessage
from langchain.tools import tool, ToolRuntime
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command
from pprint import pprint

load_dotenv()

# %%
@tool
def read_email(runtime: ToolRuntime) -> str:
    """Read an email from the given address."""
    # take email from state
    return runtime.state["email"]

@tool
def send_email(body: str) -> str:
    """Send an email to the given address with the given subject and body."""
    # fake email sending
    return "Email sent"
```

```

# %%
class EmailState(AgentState):
    email: str

agent = create_agent(
    model="gpt-5-nano",
    tools=[read_email, send_email],
    state_schema=EmailState,
    checkpointer=InMemorySaver(),
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "read_email": False,
                "send_email": True,
            },
            description_prefix="Tool execution requires approval",
        ),
    ],
)

```

```

# %%
config = {"configurable": {"thread_id": "1"}}

```

```
response = agent.invoke(  
    {  
        "messages": [  
            HumanMessage(content="Please read my email and send a response immediately. Send the reply now in the same thread.")  
        ],  
        "email": "Hi Se^in, I'm going to be late for our meeting tomorrow. Can we reschedule? Best, John."  
    },  
    config=config,  
)
```

```
# %%  
pprint(response)
```

```
# %%  
print(response['__interrupt__'])
```

```
# %%  
# Access just the 'body' argument from the tool call  
print(response['__interrupt__'][0].value['action_requests'][0]['args']['body'])
```

```
# ## Approve
```

```
# %%
```

```
response = agent.invoke(  
    Command(  
        resume={"decisions": [{"type": "approve"}]}  
    ),  
    config=config, # Same thread ID to resume the paused conversation  
)
```

```
# %%
```

```
pprint(response)
```

```
# ## Reject
```

```
# %%
```

```
response = agent.invoke(  
    Command(  
        resume={  
            "decisions": [  
                {  
                    "type": "reject",  
                    # An explanation of why the request was rejected
```

```

        "message": "No please sign off - Your merciful leader, Se^in
    ."
    }
    ]
    }
    ),
    config=config, # Same thread ID to resume the paused conversation
)

# %%
pprint(response)

# %%
print(response['__interrupt__'][0].value['action_requests'][0]['args']['body'])

# ## Edit

# %%
response = agent.invoke(
    Command(
        resume={
            "decisions": [
                {

```

```

        "type": "edit",
        # Edited action with tool name and args
        "edited_action": {
            # Tool name to call.
            # Will usually be the same as the original action.
            "name": "send_email",
            # Arguments to pass to the tool.
            "args": {"body": "This is the last straw, you're fired!"
        },
    },
    ],
},
),
config=config, # Same thread ID to resume the paused conversation
)

# %%
pprint(response)

```

3_4_dynamic_models.py

```
from typing import Callable

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from langchain.chat_models import init_chat_model
from langchain.messages import AIMessage, HumanMessage

load_dotenv()

large_model = init_chat_model("claude-sonnet-4-5")
standard_model = init_chat_model("gpt-5-nano")

@wrap_model_call
def state_based_model(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """Select model based on State conversation length."""
    # request.messages is a shortcut for request.state["messages"]
    message_count = len(request.messages)
```

```

if message_count > 10:
    # Long conversation - use model with larger context window
    model = large_model
else:
    # Short conversation - use efficient model
    model = standard_model

request = request.override(model=model)

return handler(request)

agent = create_agent(
    model="gpt-5-nano",
    middleware=[state_based_model],
    system_prompt="You are roleplaying a real life helpful office intern."
)

response = agent.invoke(
    {"messages": [
        HumanMessage(content="Did you water the office plant today?")
    ]}
)

print(response["messages"][-1].content)

# %%

```

```

print(response["messages"][-1].response_metadata["model_name"])

response = agent.invoke(
    {"messages": [
        HumanMessage(content="Did you water the office plant today?"),
        AIMessage(content="Yes, I gave it a light watering this morning."),
        HumanMessage(content="Has it grown much this week?"),
        AIMessage(content="It's sprouted two new leaves since Monday."),
        HumanMessage(content="Are the leaves still turning yellow on the edges?"),
        AIMessage(content="A little, but it's looking healthier overall."),
        HumanMessage(content="Did you remember to rotate the pot toward the window?"),
        AIMessage(content="I rotated it a quarter turn so it gets more even light."),
        HumanMessage(content="How often should we be fertilizing this plant?"),
        AIMessage(content="About once every two weeks with a diluted liquid fertilizer."),
        HumanMessage(content="When should we expect to have to replace the pot?")
    ]}
)

print(response["messages"][-1].content)

# %%

```

```
print(response["messages"][-1].response_metadata["model_name"])
```

```
# %%
```

3_4_dynamic_prompts.py

```
from dataclasses import dataclass

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest
from langchain.messages import HumanMessage

load_dotenv()

@dataclass
class LanguageContext:
    user_language: str = "English"

@dynamic_prompt
def user_language_prompt(request: ModelRequest) -> str:
    """Generate system prompt based on user role."""
    user_language = request.runtime.context.user_language
    base_prompt = "You are a helpful assistant."

    if user_language != "English":
        return f"{base_prompt} only respond in {user_language}."
    elif user_language == "English":
        return base_prompt
```

```
agent = create_agent(  
    model="gpt-5-nano",  
    context_schema=LanguageContext,  
    middleware=[user_language_prompt]  
)  
  
response = agent.invoke(  
    {"messages": [HumanMessage(content="Hello, how are you?")]},  
    context=LanguageContext(user_language="Irish")  
)  
  
print(response["messages"][-1].content)  
  
response = agent.invoke(  
    {"messages": [HumanMessage(content="Hello, how are you?")]},  
    context=LanguageContext(user_language="Spanish")  
)  
  
print(response["messages"][-1].content)  
  
response = agent.invoke(  
    {"messages": [HumanMessage(content="Hello, how are you?")]},  
    context=LanguageContext(user_language="French")  
)  
  
print(response["messages"][-1].content)
```


3_4_dynamic_tools.py

```
from dataclasses import dataclass
from typing import Callable, Dict, Any

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse
from langchain.messages import HumanMessage
from langchain.tools import tool
from langchain_community.utilities import SQLDatabase
from tavily import TavilyClient

load_dotenv()

tavily_client = TavilyClient()

db = SQLDatabase.from_uri("sqlite:///resources/Chinook.db")

@tool
def web_search(query: str) -> Dict[str, Any]:
    """Search the web for information"""
```

```

    return tavily_client.search(query)

@tool
def sql_query(query: str) -> str:

    """Obtain information from the database using SQL queries"""

    try:
        return db.run(query)
    except Exception as e:
        return f"Error: {e}"

@dataclass
class UserRole:
    user_role: str = "external"

@wrap_model_call
def dynamic_tool_call(request: ModelRequest,
                      handler: Callable[[ModelRequest], ModelResponse]) -> ModelResponse:

    """Dynamically call tools based on the runtime context"""

    user_role = request.runtime.context.user_role

```

```

if user_role == "internal":
    pass # internal users get access to all tools
else:
    tools = [web_search] # external users only get access to web search
    request = request.override(tools=tools)

return handler(request)

agent = create_agent(
    model="gpt-5-nano",
    tools=[web_search, sql_query],
    middleware=[dynamic_tool_call],
    context_schema=UserRole
)

response = agent.invoke(
    {"messages": [HumanMessage(content="How many artists are in the database?")
    ]},
    context={"user_role": "external"}
)

print(response["messages"][-1].content)

# %%

```

3_5_email_agent.py

```
from dataclasses import dataclass
from typing import Callable

from dotenv import load_dotenv
from langchain.agents import AgentState, create_agent
from langchain.agents.middleware import (
    HumanInTheLoopMiddleware,
    ModelRequest,
    ModelResponse,
    dynamic_prompt,
    wrap_model_call,
)
from langchain.messages import HumanMessage, ToolMessage
from langchain.tools import tool, ToolRuntime
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command
from pprint import pprint

load_dotenv()

# email agent
# - authenticates user
# - only then are they allowed into the "inbox"
```

```
# - dynamic tools and prompt on the condition of there being an email and p
password in state that match hardcoded
# - checks "inbox"
# - email in tool
# - sends emails
# - human in the loop
```

```
@dataclass
class EmailContext:
    email_address: str = "julie@example.com"
    password: str = "password123"
```

```
class AuthenticatedState(AgentState):
    authenticated: bool
```

```
@tool
def check_inbox() -> str:
    """Check the inbox for recent emails"""
    return """
    Hi Julie,
    I'm going to be in town next week and was wondering if we could grab a coff
    ee?
    - best, Jane (jane@example.com)"""
```

```
"""
```

```
@tool
```

```
def send_email(to: str, subject: str, body: str) -> str:
    """Send an response email"""
    return f"Email sent to {to} with subject {subject} and body {body}"
```

```
@tool
```

```
def authenticate(email: str, password: str, runtime: ToolRuntime) -> Command:
    """Authenticate the user with the given email and password"""
    if email == runtime.context.email_address and password == runtime.context.p
password:
        return Command(
            update={
                "authenticated": True,
                "messages": [
                    ToolMessage("Successfully authenticated", tool_call_id=runt
ime.tool_call_id)
                ],
            }
        )
    else:
        return Command(
            update={
```

```

        "authenticated": False,
        "messages": [
            ToolMessage("Authentication failed", tool_call_id=runtime.t
ool_call_id)
        ],
    }
)

```

```

authenticated_prompt = "You are a helpful assistant that can check the inbox an
d send emails."
unauthenticated_prompt = "You are a helpful assistant that can authenticate use
rs."

```

```

@wrap_model_call
def dynamic_tool_call(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:

    """Allow read inbox and send email tools only if user provides correct email
    and password"""

    authenticated = request.state.get("authenticated")

```

```

if authenticated:
    tools = [check_inbox, send_email]
else:
    tools = [authenticate]

request = request.override(tools=tools)
return handler(request)

```

```

@dynamic_prompt
def dynamic_prompt(request: ModelRequest) -> str:
    """Generate system prompt based on authentication status"""
    authenticated = request.state.get("authenticated")

    if authenticated:
        return authenticated_prompt
    else:
        return unauthenticated_prompt

```

```

agent = create_agent(
    "gpt-5-nano",
    tools=[authenticate, check_inbox, send_email],
    checkpoint=InMemorySaver(),
    state_schema=AuthenticatedState,
    context_schema=EmailContext,
)

```

```

middleware=[
    dynamic_tool_call,
    dynamic_prompt,
    HumanInTheLoopMiddleware(
        interrupt_on={
            "authenticate": False,
            "check_inbox": False,
            "send_email": True,
        }
    ),
],
)

config = {"configurable": {"thread_id": "1"}}

response = agent.invoke(
    {"messages": [HumanMessage(content="draft 1")]},
    context=EmailContext(),
    config=config,
)

print(response["messages"][-1].content)

# %%
print(response["__interrupt__"][0].value["action_requests"][0]["args"]["body"])

```

```
# %%
response = agent.invoke(
    Command(
        resume={"decisions": [{"type": "approve"}]} # or "reject"
    ),
    config=config, # Same thread ID to resume the paused conversation
)

print(response["messages"][-1].content)

# %%
pprint(response)

# %%
```

3_6_bonus_rag.py

```
from dotenv import load_dotenv
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_core.vectorstores import InMemoryVectorStore
from langchain.tools import tool
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from pprint import pprint

load_dotenv()

# ## Semantic search

loader = PyPDFLoader("resources/acmecorp-employee-handbook.pdf")

data = loader.load()

print(data)

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200, add_start_index=True
)
all_splits = text_splitter.split_documents(data)
```

```

print(len(all_splits))

embeddings = OpenAIEmbeddings(model="text-embedding-3-large")

vector_store = InMemoryVectorStore(embeddings)

ids = vector_store.add_documents(documents=all_splits)

results = vector_store.similarity_search(
    "How many days of vacation does an employee get in their first year?"
)

print(results[0])

# ## RAG Agent

@tool
def search_handbook(query: str) -> str:
    """Search the employee handbook for information"""
    results = vector_store.similarity_search(query)
    return results[0].page_content

agent = create_agent(
    model="gpt-5-nano",
    tools=[search_handbook],

```

```
    system_prompt="You are a helpful agent that can search the employee handbook for information."  
)  
  
response = agent.invoke(  
    {"messages": [HumanMessage(content="How many days of vacation does an employee get in their first year?")]}  
)  
  
pprint(response)
```

3_7_bonus_sql.py

```
from dotenv import load_dotenv
from langchain_community.utilities import SQLDatabase
from langchain.tools import tool
from langchain.agents import create_agent
from langchain.messages import HumanMessage
from pprint import pprint

load_dotenv()

db = SQLDatabase.from_uri("sqlite:///resources/Chinook.db")

@tool
def sql_query(query: str) -> str:
    """Obtain information from the database using SQL queries"""

    try:
        return db.run(query)
    except Exception as e:
        return f"Error: {e}"
```

```
sql_query.invoke("SELECT * FROM Artist LIMIT 10")

agent = create_agent(
    model="gpt-5-nano",
    tools=[sql_query]
)

question = HumanMessage(content="Who is the most popular artist beginning with
'S' in this database?")

response = agent.invoke(
    {"messages": [question]}
)

pprint(response['messages'])

print(response["messages"][-3].tool_calls[0]['args']['query'])

# %%
```